Square-Root Circuit Bounds for Iterated Functions

Samuel Schlesinger

October 16, 2026

Preliminary draft

Abstract

Let $F : \{0,1\}^n \to \{0,1\}^n$ be computed by a Boolean circuit of size s, and write $A = s + n^3$. We prove that every fixed iterate F^T, for an arbitrary integer $T \geq 0$, has a bounded-fan-in Boolean circuit of size

$$O\left(A + \sqrt{n^3 2^n A}\right).$$

The circuit is nonuniform in F and T, but the bound is independent of the magnitude and bit length of T. No injectivity assumption is required. Consequently, all fixed iterates of a polynomial-size update rule have circuits of size $2^{n/2+O(\log n)}$. The same exponential bound holds when an n-bit iteration count is part of the input. We also give a batched construction that shares preprocessing across independent trajectories.

The proof combines sparse checkpoints in a functional graph with a forward-only correction identity and batched dictionary lookup implemented by sorting networks. All lookup and selection gates are included in the size bound. The theorem recovers square-root-size circuits for permutation inversion, consistently with possible one-wayness against polynomial-size adversaries. Finally, for even n we construct an explicit linear-size permutation on n bits for which some fixed power requires $\Omega(2^{n/2}/n)$ gates. Such hard powers necessarily have binary exponents of length $\Omega(2^{n/2}/n^2)$. The counting argument neither identifies an explicit hard power nor establishes one-way functions. We also compute canonical cycle labels, reachability, and first hitting times. For permutations differing from a counter at r states, all signed fixed powers have size $O(n(r+1))$, and an m-bit input time costs $O(n(r+m+1))$. A Lean formalization checks the fixed-power, variable-time, batched, inversion, and navigation circuits and the matching-exponent lower bound, together with the sparse-counter fixed-power bound. It also treats finite invariant sets with explicit bit encodings in arbitrary universes.

Keywords. Boolean circuit complexity; function iteration; functional graphs; nonuniform computation; sorting networks; permutation inversion; one-way functions.

1 Introduction

A simple transition rule can describe a computation of enormous duration. How complicated can its final-state map be as a Boolean function of the initial state? We study this question for a fixed map $F : \{0,1\}^n \to \{0,1\}^n$. The computational resource is the total number of Boolean gates, rather than the number of evaluations of F, the number of memory probes, or the time of a random-access algorithm.

Let $N = 2^n$ and suppose that F has a circuit of size s. Directly unrolling T applications gives a circuit of size at most Ts for F^T. Alternatively, hardwiring a complete truth table and testing

the input against all N states gives an $O(nN)$-gate circuit, independently of T. The latter bound ignores the possibility that F itself is much simpler than an arbitrary map on its state space. Our main result exploits that possibility.

Theorem 1.1 (Fixed-power tradeoff)**.** *Let $n \geq 1$, let $F : \{0,1\}^n \to \{0,1\}^n$ satisfy $\mathrm{C}(F) \leq s$, and put $N = 2^n$ and $A = s + n^3$. For every integer $T \geq 0$ and every integer $1 \leq L \leq N$,*

$$\mathrm{C}(F^T) = O\Big(Ls + n^3\left(L + \frac{N}{L}\right)\Big). \tag{1.1}$$

In particular,

$$\mathrm{C}(F^T) = O\Big(A + \sqrt{n^3 NA}\Big). \tag{1.2}$$

The implicit constants are absolute. The circuit may depend on F and T, but the size bound does not depend on the bit length of T.

Here and throughout, circuits use the full fan-in-two Boolean basis and have unrestricted fan-out. The results also hold over the basis $\{\mathsf{AND}, \mathsf{OR}, \mathsf{NOT}\}$ with a constant-factor change. Circuit size refers to the whole multi-output circuit, not to a separate size bound for each output bit.

Corollary 1.2. *If $\mathrm{C}(F_n) = n^{O(1)}$, then*

$$\sup_{T \geq 0} \mathrm{C}(F_n^T) \leq 2^{n/2 + O(\log n)}. \tag{1.3}$$

The constant hidden in $O(\log n)$ may depend on the polynomial bound for $\mathrm{C}(F_n)$, but not on T.

The quantifiers in Theorem 1.1 are important. For each F and T we construct a circuit that works for *every* initial state. We do not construct a single circuit taking an arbitrary-length exponent as input. Nevertheless, an m-bit exponent can be supplied as input at a multiplicative cost of m:

$$\mathrm{C}((x,t) \mapsto F^t(x)) = O\Big(m\left(A + \sqrt{n^3 NA}\right)\Big), \qquad t \in \{0,1\}^m. \tag{1.4}$$

We prove this in Section 5.1. In particular, $m = n$ preserves the exponent in Equation (1.3). We also give a shared-dictionary construction for r independent initial states, replacing the main square-root term $r\sqrt{n^3 NA}$ of separate constructions by $\sqrt{rn^3 NA}$; see Section 5.2.

For arbitrary fixed powers, the exponent $1/2$ cannot be improved in general. In Section 8 we give an explicit family of permutations P_n with $\mathrm{C}(P_n) = O(n)$ and

$$\max_{T \geq 0} \mathrm{C}(P_n^T) = 2^{n/2 + O(\log n)} \tag{1.5}$$

for even n. The hard exponent is obtained by counting over a very large cyclic group of powers. This is not an explicit exponential circuit lower bound: the permutation is explicit, but the hard power is not specified by an efficient procedure. Moreover, the same permutation has an easy iterator when the time parameter has polynomially many bits.

For a permutation P, choosing $T = \mathrm{ord}(P) - 1$ recovers a circuit for P^{-1}. Thus every polynomial-size permutation on n bits admits an exact inverter of size $2^{n/2 + O(\log n)}$. This exponential upper bound is compatible with one-wayness against polynomial-size circuits. We explain this consequence and its limitations in Section 9.

1.1 Proof overview

The functional graph of F consists of directed cycles with directed trees feeding into them. For a parameter L, we select $O(N/L)$ checkpoints so that every trajectory either encounters one within $2L$ steps or repeats a state within that many steps. A repeated state reveals a period and permits direct recovery of the answer from the initial trajectory.

Set $K = 2L$ and assume $T \geq K$. At each checkpoint h, store $z_h = F^{T-K}(h)$. If the initial trajectory encounters h at time $d \leq K$, then

$$F^{K-d}(z_h) = F^{K-d}\big(F^{T-K}(F^d(x))\big) = F^T(x). \tag{1.6}$$

Thus only a short *forward* correction is needed after the lookup. There is no inverse operation, so the argument applies to noninjective maps.

A separate lookup for each of $\Theta(L)$ visited states against $\Theta(N/L)$ checkpoints would cost $\Theta(N)$ comparisons. Instead, we generate the full initial trajectory before looking anything up and resolve all its queries together. Sorting the queries and dictionary records reduces this cost to $\mathrm{poly}(n)(L + N/L)$ Boolean gates. Combining the initial and final short simulations with this batched lookup gives Equation (1.1).

1.2 Related work and status of the formulation

Preprocessing trajectories to exchange storage for later computation is classical. Hellman's cryptanalytic time–memory tradeoff is an early example of this approach [2]. More directly, Munro, Raman, Raman, and Rao [3, Section 3.1] give a shortcut representation for permutation inversion: with a parameter L, their black-box construction uses $O(L)$ evaluations and $O((N/L)(\log N + \log L))$ auxiliary bits. They also give succinct representations of general functions supporting positive powers with a word-size exponent [3, Section 5]. Those results concern representation space, RAM time, and black-box access, rather than total Boolean gate count.

A closer circuit antecedent is Hirahara, Ilango, and Williams [4, Section 3.2]: they obtain square-root-size permutation-inversion circuits using checkpoints and batched lookup. Their lookup theorem builds on a lemma of Paul. This draft generalizes that permutation-inversion construction to arbitrary fixed forward powers of possibly noninjective maps. It does not subsume their general-function inversion and compression theorems.

Eppstein [5] studies uniform iteration of polynomial-time bijections, with the iteration count supplied in binary, and characterizes the resulting class of functional problems as $\mathsf{FP}^{\mathsf{PSPACE}}$. Our resource question differs: the map is fixed, the circuits are nonuniform, and the size bound remains exponential in the state length. Our result supplies no polynomial-time algorithm for that iteration problem.

The present construction uses the same checkpoint and batched-lookup principles. Batcher's sorting networks [1] implement the lookup, while the shifted payload in Equation (1.6) and the checkpoint-or-repetition lemma handle general powers and functional graphs. No RAM memory probe is treated as a unit-cost circuit operation. Checkpointing, sorting-based lookup, and square-root-size permutation inversion are not claimed as new. Historical priority for the full fixed-power theorem and the shared bounds in this draft remains unresolved.

2 Model and circuit primitives

2.1 Conventions

All logarithms are to base two. The notation F^t denotes t-fold composition, with F^0 equal to the identity. The quantity $\mathrm{C}(G)$ is the minimum number of internal gates in an acyclic Boolean circuit computing all output bits of G. Each internal gate computes a Boolean function of at most two bits. Inputs, wires, fan-out, and output designations are not charged as gates. The two constant bits are available; charging a constant number of gates for them would not change any bound.

The circuit for an iterate is allowed to depend on the entire function F and on its fixed exponent. Preprocessing time and the time required to find the circuit are not bounded. Hardwired data do not provide free addressing: every comparison, multiplexer, and routing operation used to access them is realized with Boolean gates and included in our estimates.

All state words have n bits. Because $L \le 2^n$, indices up to $2L$ fit in $n+2$ bits. In the batched construction $r \le 2^n$, so query identifiers and record positions also have $O(n)$ bits. We retain explicit polynomial factors in n for a concrete implementation; these factors are not claimed to be optimal.

2.2 Arithmetic and selection

Comparison, addition, subtraction, and conditional selection of w-bit words have $O(w)$-gate circuits. Binary long division gives an $O(w^2)$-gate circuit for the remainder of a w-bit integer modulo a positive w-bit integer: process the dividend bits from most to least significant, and at each stage conditionally subtract the modulus from the current shifted remainder. There are $O(w)$ stages and $O(w)$ gates per stage.

A circuit can select one of q words of width w using $O(qw)$ gates by a binary tree of word multiplexers, padding to a power of two when necessary. It can also select one flagged record from an ordered list of q records in $O(qw)$ gates. For example, a left-to-right scan maintains a bit indicating whether a flagged record has already been found and retains the first one. This scan is simply an acyclic chain of gates; no depth bound is needed here.

Lemma 2.1 (Batched fixed-dictionary lookup)**.** *Suppose a fixed dictionary consists of M distinct w-bit keys, each with a w-bit payload. Given $Q \ge 1$ query keys, a Boolean circuit can output a match bit and the corresponding payload for every query, in the original query order, using*

$$O\Big((M+Q)(w+\log(M+Q+2))\log^2(M+Q+2)\Big) \tag{2.1}$$

gates. The payload of an unmatched query can be set to zero. In particular, if $w = O(n)$ and $\log(M+Q+2) = O(n)$, the cost is $O(n^3(M+Q))$.

Proof. Let $R = M+Q$. Form one record for each dictionary entry and one for each query. A record contains a key, a dictionary/query tag, a payload field, and an original-position field. The record width is $O(w+\log(R+2))$.

Sort the records lexicographically by key, then by the tag with dictionary records first, and then by original position. A sorting network with $O(R\log^2(R+2))$ comparators exists [1]. A comparator and the resulting conditional swap of the entire records cost $O(w+\log(R+2))$ Boolean gates. Padding uses a separate dummy-record bit so that no actual key is reserved as a sentinel.

In the sorted list, a dictionary record precedes every query with its key. Scan the list, resetting the match bit when the key changes and propagating the payload from a dictionary record through the equal-key run that follows it. If an equal-key run has no dictionary record, every query in that

run receives match bit zero. Dictionary keys are distinct, so no run contains two competing payloads. This scan uses $O(R(w + \log(R + 2)))$ gates.

Finally, sort records by a tag that places queries first and then by the query's original position. The first Q records give the requested outputs in order. A second sorting network has the same asymptotic cost as the first. Every operation described is a fixed acyclic circuit. □

Lemma 2.2 (Detecting a repeated state)**.** *Given Q words of w bits, a circuit can return a flag indicating whether two words are equal and, when the flag is one, indices $i < j$ of two equal words. Its size is*

$$O\Big(Q(w + \log(Q + 2)) \log^2(Q + 2)\Big).$$

In particular, for $Q \le 2^{n+O(1)}$ and $w = O(n)$, the size is $O(n^3 Q)$.

Proof. Attach each word's original index and sort by word and then by index. A duplicate exists exactly when two adjacent sorted records have the same word. Compare adjacent words and select one flagged pair. Sorting by index within an equal-word run guarantees that the returned indices satisfy $i < j$. The remaining comparisons and selection use at most $O(Q(w + \log(Q + 2)))$ gates. □

3 Sparse checkpoints in functional graphs

The functional graph of a map F has one vertex for each state and the edge $v \to F(v)$ at every vertex. Each weakly connected component has a unique directed cycle, with directed trees feeding into it. For a noncycle vertex, its *depth* is the number of steps needed to reach the cycle.

Lemma 3.1 (Checkpoint or repetition)**.** *Let F be a function on a set of N states, and let $1 \le L \le N$. There is a set H of at most $\lfloor N/L \rfloor$ states such that, for every x, either two of $x, F(x), \dots, F^{L-1}(x)$ are equal or $F^d(x) \in H$ for some $d < 2L$. More generally, the same conclusion holds for starting states in any finite forward-invariant set S, with $|H|L \le |S|$ and $H \subseteq S$. The ambient state type need not be finite.*

Proof. Choose a maximal family of pairwise vertex-disjoint simple directed paths, each consisting of L vertices in S. Put the last vertex of each path into H. Disjointness gives $|H|L \le |S|$. If the first L states of an orbit are distinct, they form such a path in S. By maximality this path meets a chosen path: say $F^i(x) = F^j(a)$, where $i, j < L$ and the chosen path starts at a. Its marked endpoint is reached from x after $i + (L - 1 - j) < 2L$ steps. If the prefix is not simple, it already contains a repetition. Taking S to be the entire state space proves the first claim. The Lean proof implements the same packing argument by induction on $|S|$, removing one simple path at a time. □

Remark 3.2. It is essential that the lemma allows a short repetition instead of a checkpoint. A function can have N fixed points, in which case a set meeting every eventual cycle must contain all N states. Such cycles need no stored shortcut: their behavior is already visible locally.

4 Circuits for fixed powers

We now prove Theorem 1.1. The construction is uniform as a description of a circuit from the chosen constants, but no efficient procedure for computing those constants is asserted.

4.1 Short trajectories and shifted payloads

Fix L and let $K = 2L$. If $T < K$, use T copies of a circuit for F; this costs at most $2Ls$ gates. Hence assume $T \geq K$. Choose H as in Lemma 3.1. For every $h \in H$, hardwire the n-bit payload

$$z_h = F^{T-K}(h). \tag{4.1}$$

The dictionary is fixed independently of the input x and has at most N/L entries.

On input x, first compute

$$x_0 = x, \quad x_1 = F(x), \quad \ldots, \quad x_K = F^K(x). \tag{4.2}$$

This requires Ks gates. Apply Lemma 2.1 to all $K+1$ states in Equation (4.2), using the dictionary $h \mapsto z_h$. Select one matching record, if any, and retain its original trajectory index d. This gives a match flag and a pair (d, z_h) with $x_d = h$ whenever the flag is one.

For the matched case, compute a second short trajectory

$$z_h, F(z_h), \ldots, F^K(z_h)$$

and select its entry at position $K - d$. Its value is

$$\begin{aligned} F^{K-d}(z_h) &= F^{K-d}\big(F^{T-K}(h)\big) \\ &= F^{T-d}(h) \\ &= F^{T-d}\big(F^d(x)\big) = F^T(x). \end{aligned} \tag{4.3}$$

All exponents used here are nonnegative. This second trajectory costs another Ks gates; selection costs $O(nK)$ gates.

4.2 The repeated-state case and very large exponents

Use Lemma 2.2 on Equation (4.2). If no checkpoint was found, Lemma 3.1 guarantees indices

$$0 \leq i < j \leq K, \qquad x_i = x_j.$$

Put $p = j - i$. Determinism implies $x_{u+p} = x_u$ for all $u \geq i$. The number p need not be the least period. Since $T \geq K \geq i$,

$$F^T(x) = x_{\,i+((T-i) \bmod p)}. \tag{4.4}$$

The selected index is at most $i + p - 1 = j - 1$, so the required state already occurs in the initial trajectory.

To implement Equation (4.4) without paying for the bit length of T, hardwire the table

$$R_T(p) = T \bmod p, \qquad 1 \leq p \leq K. \tag{4.5}$$

There are K entries, each of $O(n)$ bits. A multiplexer tree retrieves $R_T(p)$ in $O(nK)$ gates. Compute $i \bmod p$ by binary long division and then compute

$$(T - i) \bmod p = \big(R_T(p) - (i \bmod p)\big) \bmod p$$

using a comparison, subtraction, and a conditional addition of p. These operations take $O(n^2)$ gates. A further $O(nK)$ gates select the state prescribed in Equation (4.4).

A circuit evaluates both branches, so all intermediate operations must also be defined when their output will be discarded. If a repeated pair is absent, supply the dummy values $i = 0$ and $p = 1$ to the arithmetic branch. If a checkpoint is absent, supply a dummy state and $d = 0$ to the second trajectory. The final output selects the checkpoint branch when its match flag is one, and otherwise selects the repetition branch. The checkpoint lemma ensures that the selected branch is always valid. In particular, no division by zero occurs on any input.

4.3 Size bound

The two simulations together use at most $2Ks = 4Ls$ gates. The dictionary and query list contain

$$|H| + (K+1) = O\left(\frac{N}{L} + L\right)$$

records of $O(n)$ bits, and the logarithm of this number is $O(n)$. Thus Lemma 2.1 costs $O(n^3(N/L+L))$ gates. Repeated-state detection costs $O(n^3L)$ gates, and all remaining lookup, arithmetic, and selection operations cost $O(nL + n^2)$ gates. Since $n, L \geq 1$, the latter cost is absorbed by $O(n^3L)$. This proves Equation (1.1).

Write the tradeoff as $O(AL + n^3N/L)$, where $A = s + n^3$, and choose

$$L = \max\left\{1, \left\lceil\sqrt{n^3N/A}\right\rceil\right\}. \tag{4.6}$$

Since $A \geq n^3$, this choice satisfies $L \leq \left\lceil\sqrt{N}\right\rceil \leq N$. If $A \leq n^3N$, then $\sqrt{n^3N/A} \leq L \leq 2\sqrt{n^3N/A}$. Hence

$$AL = O(\sqrt{n^3NA}), \qquad n^3N/L = O(\sqrt{n^3NA}).$$

If $A > n^3N$, the choice is $L = 1$ and the cost is $O(A)$. This establishes Equation (1.2) and completes the proof of Theorem 1.1. □

Remark 4.1 (Where the exponent is stored). For $T \geq K$, the information about T used by the circuit is confined to the checkpoint payloads in Equation (4.1) and the residue table in Equation (4.5). Their total length is $O(n(N/L + L))$ bits. The construction does not place the binary expansion of T on a long hidden input tape, and it does not perform uncharged arithmetic on an arbitrarily large integer. Computing the hardwired information is a preprocessing task outside the asserted circuit bound.

4.4 Invariant sets and image-sensitive bounds

Theorem 4.2 (Invariant-set tradeoff)**.** *Let $S \subseteq \{0,1\}^n$ be forward invariant under F, with $|S| = M$. For every fixed $T \geq 0$ and $1 \leq L \leq N$, there is a circuit agreeing with F^T on S of size*

$$O\left(Ls + n^3\left(L + \frac{M}{L}\right)\right).$$

The circuit is total on $\{0,1\}^n$; correctness outside S is not asserted.

Proof. Use the invariant-set version of Lemma 3.1, and otherwise apply the fixed-power construction unchanged. A state still occupies n bits, but the dictionary contains only $O(M/L)$ records. □

Corollary 4.3 (Image sensitivity)**.** *Put $M_d = |\operatorname{im}(F^d)|$. For $T \geq d$ and $1 \leq L \leq M_d$,*

$$\mathrm{C}(F^T) \leq \mathrm{C}(F^d) + O\left(Ls + n^3\left(L + \frac{M_d}{L}\right)\right).$$

Consequently this is at most $\mathrm{C}(F^d) + O(A + \sqrt{n^3M_dA})$.

Proof. The image of F^d is forward invariant, since $F(F^d(x)) = F^d(F(x))$. Compute $F^d(x)$ and apply Theorem 4.2 on its image for $T - d$ more steps. Optimize L as in Equation (4.6), replacing N by M_d. □

5 Input times and shared evaluation

5.1 An iteration count supplied as input

Definition 5.1. For $m \geq 1$, define

$$\mathrm{Iter}_{F,m} : \{0,1\}^n \times \{0,1\}^m \longrightarrow \{0,1\}^n, \qquad \mathrm{Iter}_{F,m}(x,t) = F^{\mathrm{val}(t)}(x),$$

where $\mathrm{val}(t)$ is the nonnegative integer encoded by t.

Theorem 5.2 (Variable-time iteration)**.** *Under the assumptions of Theorem 1.1, for every $m \geq 1$,*

$$\mathrm{C}(\mathrm{Iter}_{F,m}) = O\Big(m\left(A + \sqrt{n^3 NA}\right)\Big). \tag{5.1}$$

In particular,

$$\mathrm{C}(\mathrm{Iter}_{F,n}) = O\Big(n\left(A + \sqrt{n^3 NA}\right)\Big).$$

If s and m are polynomial in n, then $\mathrm{C}(\mathrm{Iter}_{F,m}) \leq 2^{n/2+O(\log n)}$.

Proof. Write $t = \sum_{j=0}^{m-1} t_j 2^j$. For each j, apply Theorem 1.1 directly to the original map F and the fixed exponent 2^j, obtaining a circuit for $G_j = F^{2^j}$. Starting with $v_0 = x$, use

$$v_{j+1} = \begin{cases} G_j(v_j), & t_j = 1, \\ v_j, & t_j = 0. \end{cases}$$

Inductively, $v_j = F^{\sum_{a<j} t_a 2^a}(x)$, so $v_m = F^t(x)$. There are m circuits of the size in Equation (1.2) and m n-bit multiplexers. The extra $O(mn)$ gates are absorbed by the claimed bound. Each G_j is constructed from F using the theorem; it is not obtained by duplicating the circuit for G_{j-1}. □

Theorem 5.3 (Long input times)**.** *Write $B = A + \sqrt{n^3 NA}$. For every $m \geq 1$,*

$$\mathrm{C}(\mathrm{Iter}_{F,m}) = O(\min\{mB,\ nB + nm\}).$$

Thus the cost of processing additional time bits is additive once the state width is fixed.

Proof. The first bound is Theorem 5.2. For the second, compute the exact eventual cycle length $\ell = \ell(x)$ in $O(B)$ gates using Theorem 6.1. Replace the input time by

$$u = \begin{cases} t, & t < N, \\ N + (t - N) \bmod \ell, & t \geq N. \end{cases}$$

Then $u < 2N$ and $F^u(x) = F^t(x)$: in the second case both times are past cycle entry and have the same residue modulo ℓ.

Compute $t \bmod \ell$ by streaming its bits from most significant to least significant, retaining an $O(n)$-bit remainder. Each step doubles the remainder, adds one bit, and conditionally subtracts ℓ; a spare bit prevents overflow. Shared-carry addition and subtraction give $O(n)$ gates per input bit, including comparison and selection. The bounded modular correction by N and the final selection cost $O(n^3 + m)$ with the arithmetic circuits already used in the proof. Hence forming u costs $O(nm + n^3)$, and its $n + 1$ bits feed Theorem 5.2 at cost $O(nB)$. Since $n^3 \leq B$, the second bound follows. If $m \leq n$, one may simply use the original m-stage construction. □

5.2 Sharing a dictionary among independent trajectories

Let

$$B_{F,T,r}(x_1, \ldots, x_r) = (F^T(x_1), \ldots, F^T(x_r)).$$

The inputs $x_1, \ldots, x_r$ are independent n-bit words. The following construction uses one checkpoint dictionary for all of them.

Theorem 5.4 (Batched fixed powers)**.** *Under the assumptions of Theorem 1.1, for every* $1 \le r \le N$*,* $T \ge 0$*, and* $1 \le L \le N$*,*

$$\mathrm{C}(B_{F,T,r}) = O\Big(rLs + n^3\left(rL + \frac{N}{L}\right)\Big). \tag{5.2}$$

Consequently,

$$\mathrm{C}(B_{F,T,r}) = O\Big(rA + \sqrt{rn^3NA}\Big). \tag{5.3}$$

Proof. Set $K = 2L$. If $T < K$, unroll T copies of F independently for each input. Otherwise use the checkpoint set and dictionary from Section 4. Generate the first $K + 1$ states of all r trajectories. Together with one copy of the dictionary, these give

$$R = |H| + r(K + 1) = O(N/L + rL)$$

records. Attach the trajectory number and within-trajectory position to each query. Because $r \le N$ and $L \le N$, the record width and $\log(R + 2)$ are $O(n)$. One application of Lemma 2.1 therefore resolves all queries in $O(n^3(N/L + rL))$ gates and returns them in their original order.

Select a matching checkpoint separately within each trajectory. Run repeated-state detection separately on each trajectory as well; this costs $O(rn^3L)$ gates and does not confuse a collision between two different trajectories with a repetition along one trajectory. The correction trajectory and output selection are also performed separately for each input. The fixed residue table may be implemented with a separate $O(nL)$-gate lookup for each trajectory, whose total cost is absorbed by $O(rn^3L)$. The two simulation phases together cost $O(rLs)$ gates. This proves Equation (5.2).

Choose

$$L = \max\left\{1, \left\lceil \sqrt{n^3N/(rA)} \right\rceil \right\}.$$

As before $1 \le L \le N$. If $rA \le n^3N$, then $rAL = O(\sqrt{rn^3NA})$ and $n^3N/L = O(\sqrt{rn^3NA})$. If $rA > n^3N$, the choice is $L = 1$ and the cost is $O(rA)$. This gives Equation (5.3). □

Corollary 5.5 (Independent input times)**.** *For* $1 \le r \le N$ *and* $m \ge 1$*, the map*

$$((x_1, t_1), \ldots, (x_r, t_r)) \longmapsto (F^{t_1}(x_1), \ldots, F^{t_r}(x_r)), \qquad t_i \in \{0,1\}^m,$$

has a circuit of size

$$O\Big(m\left(rA + \sqrt{rn^3NA}\right)\Big). \tag{5.4}$$

Proof. Use the binary-power construction of Theorem 5.2. At stage j, apply Theorem 5.4 with update map F and exponent 2^j on the r current states, then use the jth bit of each t_i to select whether to retain or update that state. The $O(mrn)$ selection gates are absorbed by Equation (5.4). □

For polynomial s and m, and $r = \lfloor 2^{\alpha n} \rfloor$ with fixed $0 \le \alpha \le 1$, Equation (5.4) is at most

$$2^{(1+\alpha)n/2 + O(\log n)}.$$

This is an upper-bound sharing statement. It is not a direct-sum separation against optimal circuits for a single trajectory.

6 Computing the Structure of Functional Graphs

Write $B = A + \sqrt{n^3 NA}$. For a state x, let $\mu(x)$ be its distance to its eventual cycle, let $\ell(x)$ be that cycle's length, and let $c(x)$ be the lexicographically least state on the cycle. Define $\phi(x) \in \{0, \ldots, \ell(x) - 1\}$ by

$$F^t(x) = F^{(\phi(x)+t) \bmod \ell(x)}(c(x)) \qquad (t \geq \mu(x)). \tag{6.1}$$

The phase is an eventual-orbit label; it does not assert that a transient state itself lies on the cycle. These labels are unique. Cycle lengths are encoded with $n+1$ bits, since a cycle may have length N.

Theorem 6.1 (Cycle labels)**.** *A circuit of size $O(B)$ computes $(\ell(x), c(x), \phi(x))$ for every x.*

Proof. Fix L and use the checkpoint construction with $K = 2L$. Hardwire the three labels for each checkpoint. If $F^d(x) = h$ for $d \leq K$, output

$$\ell(x) = \ell(h), \qquad c(x) = c(h), \qquad \phi(x) = (\phi(h) - d) \bmod \ell(h).$$

This follows by applying Equation (6.1) after both transients have ended. All records still have $O(n)$ bits.

If there is no checkpoint hit, the prefix repeats. Sort prefix records by state and then by original index. Among adjacent equal-state pairs, choose the pair with smallest second index j, retaining its first index i. This is the first repetition in the original trajectory: any earliest second occurrence is adjacent to its first occurrence in its equal-state run. The states before j are distinct, so $i = \mu(x)$ and $[i, j)$ consists of exactly one cycle traversal. Set $\ell = j - i$, and select the minimum state $c = x_p$ over $i \leq p < j$. Then $\phi(x) = (-p) \bmod \ell$. Minimum selection and the interval mask cost $O(nK)$ gates, and word arithmetic costs $O(n^2)$ gates. Sorting and lookup cost $O(n^3(L + N/L))$, while the prefix costs $O(Ls)$. Optimizing L proves the claim. Discarded branches use safe positive dummy moduli, as in the fixed-power construction. □

Theorem 6.2 (Transient depth)**.** *The exact transient depth $\mu(x)$ is computable by a circuit of size $O(B)$.*

Proof. Put $D = N!$ and $R = F^D$. Every trajectory is cyclic after N steps, every cycle length divides D, and $D \geq N$. Thus R maps into the cyclic states, fixes each such state, and satisfies $R^2 = R$. It commutes with all iterates of F, so

$$F^i(x) = F^i(R(x)) \quad \Longleftrightarrow \quad i \geq \mu(x). \tag{6.2}$$

Indeed, equality is equivalent to $F^i(x) = R(F^i(x))$, which is exactly cyclicity. The projection costs $O(B)$ gates by Theorem 1.1.

Fix a checkpoint parameter L and set $K = 2L$. Retain the first $K+1$ states of both trajectories and compare corresponding states. If the states at position K agree, select the first equality; its index is $\mu(x)$. Otherwise $\mu(x) > K$, so the original prefix has no repetition and must meet a checkpoint $h = F^d(x)$ with $d < K$. Every such hit is transient and satisfies $\mu(x) = d + \mu(h)$.

Hardwire $\mu(h)$ with each checkpoint and resolve all prefix queries using Lemma 2.1. Either branch uses $O(n)$-bit records; comparing, selecting, and adding indices cost $O(nL)$ gates. The two prefixes and shared lookup cost $O(Ls + n^3(L + N/L))$. Use the same L for the projection and these operations, then optimize as in Equation (4.6). □

Theorem 6.3 (Reachability and first hitting times)**.** *A circuit of size $O(nB)$, on input (x, y), decides whether $F^t(x) = y$ for some $t \geq 0$ and returns the least such t when it exists. The output may use a separate existence flag and n time bits.*

Proof. Compute both depths using Theorem 6.2 and both sets of cycle labels using Theorem 6.1, for a total of $O(B)$ gates. If y is transient, it is reachable exactly when

$$\mu(x) \geq \mu(y) \quad \text{and} \quad F^{\mu(x)-\mu(y)}(x) = y.$$

Indeed, transient depth decreases by exactly one per step until the cycle is reached. In this case the displayed exponent is the first hitting time. The n-bit variable-time iterator evaluates this final test in $O(nB)$ gates.

If y is cyclic, it is reachable exactly when $c(x) = c(y)$. On a positive instance its first hitting time is

$$\mu(x) + \big((\phi(y) - \phi(x) - \mu(x)) \bmod \ell(x)\big),$$

where the parenthesized residue lies in $[0, \ell(x) - 1]$. This is the first time on or after entry into the cycle with the required phase. Finally $\mu(x) + \ell(x) \leq N$, so every first hitting time is at most $N - 1$. Arithmetic may use $n + 1$ intermediate bits. The time field is unspecified on negative instances; an additional $O(n)$-gate mask can set it to zero if desired. □

Corollary 6.4 (Complete visit schedules)**.** *A circuit of size $O(nB)$, on input (x, y), returns a success flag and two integers d, p describing every visit time. A zero flag denotes no visits. For a positive flag, $d < N$ is the first visit, $0 \leq p \leq N$, and*

$$\{t \geq 0 : F^t(x) = y\} = \{d + kp : k \geq 0\}.$$

Here $p = 0$ for transient y, giving the singleton $\{d\}$, and $p = \ell(y)$ for cyclic y. The period field uses $n + 1$ bits.

Proof. Use Theorem 6.3 for the flag and d, and Theorem 6.1 and an $O(B)$ cyclicity test for p. After the first visit, a later time $t \geq d$ is a visit exactly when $F^{t-d}(y) = y$. Return times of a cyclic point are precisely the multiples of its least positive period. A transient point has no positive return. This proves the descriptor. □

For example, a visit occurs by deadline T exactly when the flag is positive and $d \leq T$. The number of visits at times $0, \ldots, T$ is

$$\begin{cases} 0, & \text{flag zero or } T < d, \\ 1, & \text{flag positive, } T \geq d,\ p = 0, \\ 1 + \lfloor (T - d)/p \rfloor, & \text{flag positive, } T \geq d,\ p > 0. \end{cases}$$

These are arithmetic consequences of the returned descriptor; the descriptor bound does not include a circuit for processing an additional deadline input of unspecified length.

Relation to navigation data structures. Function powers and path queries have established data-structure formulations [3, 7]. In particular, nearest labeled successors include reachability of a fixed singleton target. Those results charge representation space and query time in a RAM model. Here the given object is a succinct transition circuit, and every gate accessing a nonuniform table is charged. These differences specify the claim being compared; they do not establish historical novelty of the present bounds. These theorems are also checked as Boolean circuit constructions in Lean, with the implementation choices described in Section 10.

6.1 Recognizable targets and terminating computations

Corollary 6.5 (First recognizable target)**.** *Let $P : \{0,1\}^n \to \{0,1\}$ have circuit size a, and put $A_P = s + a + n^3$. Eventual target membership,*

$$H(x) = 1 \quad \Longleftrightarrow \quad \exists t \geq 0 : \ P(F^t(x)) = 1,$$

has a circuit of size $O(A_P + \sqrt{n^3 N A_P})$. Within the same bound, a circuit returns the first visited target state and its exact first visit time on positive instances.

Proof. Set $G(x) = x$ when $P(x) = 1$ and $G(x) = F(x)$ otherwise. This costs $O(s + a + n)$ gates. A first target, if one exists, is visited before step N, since an earlier repeated state would preclude a later first visit. Hence $G^N(x)$ is the first target state on positive instances, and $H(x) = P(G^N(x))$. Apply Theorem 1.1 to G. For the time, apply Theorem 6.2 to G. On a positive instance, the first target is an absorbing fixed point. A cyclic state cannot precede it on the trajectory: iteration is injective on cyclic states, so any cyclic predecessor of that fixed point is the same point. Consequently $\mu_G(x)$ is exactly the first target time. □

Corollary 6.6 (Complete-configuration simulation)**.** *Suppose a deterministic computation has its entire configuration encoded in b bits and a next-configuration circuit of size s. If halting configurations are absorbing and every valid initial configuration halts, its final-configuration map on valid initial states has circuit size*

$$O\left(s + b^3 + \sqrt{b^3 2^b (s + b^3)}\right).$$

The exact first halting time can also be returned within this bound.

Proof. Before halting, no configuration can repeat. The iterate at time 2^b is therefore the final configuration. Apply Theorem 1.1 with state width b. The first halt is the transient depth, by the same absorbing-state argument as in Corollary 6.5; use Theorem 6.2. Initialization and output decoding circuits are charged separately. □

The parameter b includes all persistent input information, control, and workspace. This is not a $2^{n/2}\,\mathrm{poly}(n)$ circuit bound for all of PSPACE as a function of input length n.

6.2 Synchronous and recurring queries

The projection $R = F^{N!}$ from the proof of Theorem 6.2 also supplies queries about the eventual dynamics.

Corollary 6.7 (Eventual synchronous coalescence)**.** *A circuit of size $O(B)$ decides, on input (x, y), whether there exists $t \geq 0$ such that $F^t(x) = F^t(y)$.*

Proof. The required equivalence is

$$\exists t \geq 0 : \ F^t(x) = F^t(y) \quad \Longleftrightarrow \quad F^N(x) = F^N(y).$$

Equality before time N persists to time N. Both states are cyclic by that time, and F restricts to a bijection on the cyclic states. Therefore later equality can be canceled back to time N. Compute two copies of F^N and compare their outputs. □

This query requires the same elapsed time on both trajectories. Orbit intersection permits different elapsed times and is a different predicate; for example, distinct points of one cycle never coalesce synchronously.

Corollary 6.8 (Recurring and permanent targets)**.** *For a fixed predicate $P : \{0,1\}^n \to \{0,1\}$ of circuit size a, each of*

$$\forall u\ \exists t \geq u :\ P(F^t(x)) = 1, \qquad \exists u\ \forall t \geq u :\ P(F^t(x)) = 1$$

is decidable by a circuit of size $O(A_P + \sqrt{n^3 N A_P})$, where $A_P = s + a + n^3$.

Proof. A target recurs arbitrarily late exactly when the eventual cycle contains an accepting state. Start at $R(x)$ and use the absorbing-target construction in Corollary 6.5. If G makes accepting states absorbing, the circuit outputs $P(G^N(R(x)))$. A successful visit is periodic and hence recurs arbitrarily late; every sufficiently late visit from x is a future visit from $R(x)$. The projection and the target circuit each obey the stated bound. Permanent eventual acceptance is the complement of recurrence of the predicate $1 - P$, which changes the bound only by a constant number of gates. □

7 Sparse modifications of a counter

The state-space parameter is not the only source of compression. Identify $\{0,1\}^n$ with $\mathbb{Z}/N\mathbb{Z}$, where $N = 2^n$, and write $R(x) = x + 1 \pmod N$. A permutation obtained by changing a small number of counter successors admits a bound linear in its sparse description, even when it is a single cycle through all N states.

Theorem 7.1 (Sparse-counter powers)**.** *Let $n \geq 1$, let F be a permutation of $\mathbb{Z}/N\mathbb{Z}$, and let $r = |\{x : F(x) \neq R(x)\}|$. For every fixed $T \in \mathbb{Z}$,*

$$\mathrm{C}(F^T) = O\big(n(r+1)\big).$$

For an unsigned m-bit time supplied as input,

$$\mathrm{C}\big((x,t) \mapsto F^t(x)\big) = O\big(n(r+m+1)\big).$$

Both constants are absolute. The dependence on time-input length is additive.

7.1 Commutation defects and displacement

The fixed-power assertion follows from a more general statement.

Proposition 7.2 (Sparse-counter centralizers)**.** *Let F be any map on $\mathbb{Z}/N\mathbb{Z}$ differing from R at r inputs. Every permutation H commuting with F has a displacement function $\delta_H(x) = H(x) - x \pmod N$ that changes on at most $2r$ counter edges. Consequently, $\mathrm{C}(H) = O(n(r+1))$.*

Proof. Put $E = \{x : F(x) \neq R(x)\}$. If $x \notin E$ and $H(x) \notin E$, then

$$H(R(x)) = H(F(x)) = F(H(x)) = R(H(x)).$$

Thus the set of commutation failures for H and R lies in $E \cup H^{-1}(E)$ and has size at most $2r$. Commutation at x is exactly $\delta_H(x+1) = \delta_H(x)$. Cutting the ordinary linear order $0, \ldots, N-1$ at these changes gives at most $2r+1$ constant intervals. Compare the input with their endpoints, select a hardwired displacement, and add it modulo N, at cost $O(n(r+1))$.

An alternative implementation, used in Lean, avoids ordering the changes. Let $D = \{i : \delta_H(i+1) \neq \delta_H(i)\}$, with cyclic addition in the arguments and ordinary integer comparisons below. Telescoping gives

$$H(x) = x + \delta_H(0) + \sum_{\substack{i \in D \\ i < x}} (\delta_H(i+1) - \delta_H(i)) \pmod{N}.$$

Each summand uses one comparison, a conditional constant word, and a linear-size adder. The wraparound edge is never selected. All gates, including the shared carry chains, are charged. □

Every signed power of a permutation commutes with it, proving the first part of Theorem 7.1. The proposition only requires injectivity of H in its cardinality argument; it does not require F to be a permutation. Its set-containment argument works on arbitrary universes, before finiteness or coordinates are imposed.

The geometry relating commutation failures to blocks in cycles is established permutation theory. Moreno and Rivera characterize these blocks in their Theorem 3.8 and Corollary 3.10 [8]. Here we use the elementary defect containment and account for the Boolean cost of the resulting representation. Historical priority of these precise circuit bounds has not been determined.

7.2 A chart constructed from the edited successors

For the live-time bound, assume first $r > 0$ and cut the counter cycle immediately after each point of E. There are r nonempty circular arcs. Inside an arc F increments; its only possible change is at the last state. Since F and R are permutations agreeing off E,

$$F(E) = R(E).$$

Indeed, the images of the unedited inputs coincide, so their complements coincide. Therefore each edited edge leads to an arc start, and these edges induce a permutation of the r arcs.

Decompose that permutation into cycles and concatenate the arcs in each cycle, retaining their individual lengths. Assign to a state the triple (c, p, ℓ) consisting of its cycle identifier, its position within that cycle, and its total cycle length. The coordinate map and its inverse are piecewise translations on the original arcs and their concatenated images. At most one original arc crosses zero; splitting it there adds one linear interval. Comparisons, shared record selection, and addition therefore implement each direction in $O(n(r+1))$ gates. Identifiers and positions use n bits, while lengths use $n+1$ bits to allow $\ell = N$. All intermediate arithmetic uses $O(n)$ bits.

In these coordinates one step preserves c, ℓ and increments p modulo ℓ. For an m-bit input t, compute its remainder by scanning bits from most significant to least significant. With current remainder $u < \ell$, the next remainder is $2u + b$ or $2u + b - \ell$, according as $2u + b < \ell$ or not. A comparison and subtraction cost $O(n)$ per bit; one extra bit prevents overflow. Add p, reduce once more, and decode. The cost is $O(nr + nm + n)$. If $r = 0$, ordinary addition modulo N gives the same bound. This proves the second part of Theorem 7.1. For fixed signed T, hardwire T mod ℓ for each cycle instead.

Given an explicit valid list of edited successors, the chart is constructible in time polynomial in $r+1$ and n. Sort the cuts, compute arc lengths, follow the induced arc permutation, and form prefix sums. Permutation preservation itself is checked by comparing edited outputs with the freed counter outputs. This procedure never enumerates the N states. Compiling a particular signed power additionally costs polynomial time in the bit length of T. A displacement table can be constructed by testing only the candidate change edges $E \cup F^{-T}(E)$, using the chart for these $O(r)$ queries.

The executable `sparse_counter.py` implements this chart and interval compiler for arbitrary positive universe sizes. An independent cycle-traversal oracle checks all 46,232 permutations on sizes 2 through 8 and all 334,381 distinct powers, as well as the one-state case, 1,000 sampled sparse permutations with signed exponents of up to 1,024 bits, and 700 binary-field examples. Additional checks on 64–256-bit universes exercise composition and inverse-coordinate identities without enumeration. These finite checks establish neither the gate bounds nor a formal compiler-correctness theorem.

7.3 Large transitions and charged coordinate changes

The improvement can apply to transitions with genuinely exponential minimum circuit size. Fix r nonempty counter arcs and reconnect them in any single cyclic order. This gives $(r-1)!$ distinct permutations, each differing from the counter at at most r states and having one orbit of length N. For $r = 2^{\lfloor \alpha n \rfloor}$ with fixed $0 < \alpha < 1$, the logarithm of this family size is $\Theta(r \log r)$. The circuit-description bound used in Section 8 implies that some members need $\Omega(r)$ gates, while all have $O(n(r+1))$-gate circuits. Consequently there exist families with

$$\mathrm{C}(F_n) = 2^{\alpha n + O(\log n)}, \qquad \sup_{T \in \mathbb{Z}} \mathrm{C}(F_n^T) = 2^{\alpha n + O(\log n)}.$$

The lower bound is existential counting, not an explicit hard transition. Using only their transition sizes in Theorem 1.1 would instead give an upper bound with exponential scale $2^{(1+\alpha)n/2}$.

Corollary 7.3 (Charged orbit coordinates)**.** *Suppose a bijection $\phi : \{0,1\}^n \to \mathbb{Z}/N\mathbb{Z}$ satisfies $\phi(G(x)) = \phi(x) + 1 \pmod N$. If a permutation F differs from G at r states, then*

$$\mathrm{C}(F^T) \le \mathrm{C}(\phi) + \mathrm{C}(\phi^{-1}) + O(n(r+1)) \quad (T \in \mathbb{Z}).$$

For an m-bit input time, the last term is $O(n(r+m+1))$.

Proof. Conjugate F by ϕ. The exceptional-set cardinality is unchanged, and the reference permutation becomes the counter. Apply Theorem 7.1 and compose with the two coordinate circuits. □

Both coordinate costs are essential. A long orbit does not itself supply cheaply computable coordinates. Conversely, these examples can visit the entire state space: the compressed object is the arrangement of arcs, without a reduction in the number of reachable states.

7.4 Sparse edits of a primitive binary-field cycle

Corollary 7.4. *Represent $K = \mathbb{F}_{2^n}$ in a fixed polynomial basis and fix a primitive element ω. If a permutation F differs from $L(x) = \omega x$ at r states, then*

$$\mathrm{C}(F^T) \le n^{O(\log n)} + O(n(r+1)) \quad (T \in \mathbb{Z}).$$

With an m-bit input time the bound is $n^{O(\log n)} + O(n(r+m+1))$.

Proof. Insert zero into the nonzero multiplicative cycle by setting $G(0) = 1$, $G(\omega^{-1}) = 0$, and $G(x) = \omega x$ elsewhere. Then G is one cycle, and F differs from it at at most $r+2$ states. Coordinates are $\phi(0) = 0$ and $\phi(\omega^j) = j + 1$ for $0 \le j < 2^n - 1$. Decoding uses finite-field exponentiation and costs $\mathrm{poly}(n)$ gates. Encoding uses discrete logarithms.

Kleinjung and Wesolowski's Theorem 1.1 [9] gives expected time $n^{O(\log n)}$ for discrete logarithms in characteristic two. To obtain exact nonuniform circuits, let $\tau(n)$ bound this expectation for every

input. Truncate each trial at $2\tau(n)$ steps and verify candidate logarithms by exponentiation. A trial succeeds with probability at least $1/2$. Among $n+1$ independent random tapes, the probability that some one of the fewer than 2^n nonzero inputs has no successful trial is less than one. Fix tapes that work for every input and select the first verified answer. Circuit simulation and verification have polynomial overhead in $\tau(n)$ and n, preserving quasipolynomial size. Apply Corollary 7.3. The case of bounded n is absorbed by the constants. □

This corollary relies on the cited discrete-logarithm theorem and the nonuniform simulation argument; those ingredients are not formalized in our Lean development. It concerns sparse permutation-preserving edits to a primitive multiplicative cycle. It does not imply the unrestricted conjectural bound $s + \widetilde{O}(2^{n/2})$.

8 A matching exponent for arbitrary fixed powers

This section gives an explicit linear-size permutation whose collection of powers is large enough that some member has circuit complexity at least $2^{n/2}/\operatorname{poly}(n)$. The counting argument is elementary and does not require an asymptotic estimate for the maximal order of a permutation.

8.1 A permutation with cycles of all lengths up to $2^{n/2} - 1$

Let $n = 2k$ and $B = 2^k - 1$. Write a state as a pair of k-bit integers (a, b). Define

$$P_n(a,b) = \begin{cases} (a, b+1 \bmod a), & 1 \le a \le B \text{ and } 0 \le b < a, \\ (a,b), & \text{otherwise.} \end{cases} \tag{8.1}$$

For each $1 \le a \le B$, the states $(a,0), (a,1), \ldots, (a,a-1)$ form one cycle of length a; every other state is fixed. Hence P_n is a permutation and

$$\operatorname{ord}(P_n) = D_B := \operatorname{lcm}(1, 2, \ldots, B). \tag{8.2}$$

Despite the notation in Equation (8.1), the update does not require a general division circuit. Compare b with a, increment b, and, when $b < a$, replace b by zero exactly when the increment equals a. Otherwise retain the appropriate increment or original value. Comparison, increment, and selection each have $O(k)$-gate circuits. The first coordinate is passed through unchanged. Thus

$$\mathrm{C}(P_n) = O(n), \tag{8.3}$$

and the family has a direct polynomial-time circuit construction.

Theorem 8.1 (Powers and fixed-integer remainders)**.** *For a fixed $T \ge 0$, define $r_T(a) = T \bmod a$ when $a > 0$ and $r_T(0) = 0$, using k bits for both input and output. Then*

$$\mathrm{C}(r_T) \le \mathrm{C}(P_n^T) \le \mathrm{C}(r_T) + O(n).$$

Proof. Restrict the power circuit to inputs $(a, 0)$ and retain its second coordinate. This computes r_T, including the zero-modulus branch. Conversely, on an active state compute $b + r_T(a)$ using $k+1$ bits. Both summands are below a, so a comparison and at most one subtraction of a give the result modulo a. The spare bit retains the carry. Select the original b on inactive states and copy a to the output. All additional operations have linear-size Boolean circuits. The first inequality uses the paper's free constant-wire convention; the native Lean model charges one shared zero gate for the restriction. □

Thus the arithmetic object behind the lower bound is the remainder of a single fixed integer as a function of its input modulus. The theorem does not identify a particular hard integer.

Lemma 8.2. *For every integer $B \geq 1$,*

$$\operatorname{lcm}(1, 2, \ldots, B) \geq \frac{2^B}{B+1}. \tag{8.4}$$

Proof. We show that every binomial coefficient $\binom{B}{j}$ divides D_B. For a prime p, factorial valuations give

$$\nu_p\binom{B}{j} = \sum_{a \geq 1} \left(\left\lfloor \frac{B}{p^a} \right\rfloor - \left\lfloor \frac{j}{p^a} \right\rfloor - \left\lfloor \frac{B-j}{p^a} \right\rfloor \right).$$

Each summand is zero or one, and it is zero when $p^a > B$. Therefore

$$\nu_p\binom{B}{j} \leq \left\lfloor \log_p B \right\rfloor = \nu_p(D_B),$$

where both sides are zero for $p > B$. This proves divisibility. Since $\sum_{j=0}^{B} \binom{B}{j} = 2^B$, some binomial coefficient is at least $2^B/(B+1)$, and D_B is at least that large. □

Lemma 8.3 (Counting multi-output circuits)**.** *The number of functions $\{0,1\}^n \to \{0,1\}^n$ computable with at most q fan-in-two Boolean gates is at most*

$$(q+1)16^q(n+q+2)^{2q+n} = 2^{O((q+n)\log(n+q+2))}. \tag{8.5}$$

Proof. Topologically order the gates. At each gate choose one of at most 16 Boolean operations and two predecessors among the inputs, the two constants, and gates. Allowing all $n+q+2$ possible predecessors at each position only increases the count. Choose the n output wires from these same possibilities. Sum over gate counts from zero through q; each summand is bounded by the expression without its factor $q+1$. Different encodings may compute the same function, which only helps the upper bound. □

Theorem 8.4 (Hard powers of a simple permutation)**.** *For all sufficiently large even n, the permutation in Equation* (8.1) *satisfies*

$$\max_{T \geq 0} \mathrm{C}(P_n^T) = \Omega\left(\frac{2^{n/2}}{n}\right). \tag{8.6}$$

Together with Theorem 1.1, this gives

$$\log_2\left(\max_{T \geq 0} \mathrm{C}(P_n^T)\right) = \frac{n}{2} + O(\log n). \tag{8.7}$$

Proof. There are exactly D_B distinct powers of P_n, with representatives $0 \leq T < D_B$. By Lemma 8.2,

$$\log_2 D_B \geq B - \log_2(B+1).$$

Let $q = \lfloor cB/n \rfloor$ for a sufficiently small positive absolute constant c. Since $B = 2^{n/2} - 1$, Lemma 8.3 bounds the logarithm of the number of circuits of size at most q by

$$O((q+n)\log(n+q+2)) = O(cB + n^2).$$

Choose c so that the coefficient of B is less than $1/2$. For all sufficiently large even n, this quantity is smaller than $B - \log_2(B+1)$. Thus there are fewer size-q circuits than distinct powers of P_n. At least one power has complexity greater than q, proving Equation (8.6).

For the upper bound, substitute $\mathrm{C}(P_n) = O(n)$ into Equation (1.2). This gives $\max_T \mathrm{C}(P_n^T) = O(n^3 2^{n/2})$. Combining the upper and lower bounds and taking logarithms proves Equation (8.7). □

Theorem 8.5 (Almost every power is hard)**.** *There is an absolute $c > 0$ such that, for all sufficiently large even n,*

$$\Pr_{T \sim \mathrm{Unif}\{0,\ldots,D_B-1\}} \left[\mathrm{C}(P_n^T) \geq c \frac{2^{n/2}}{n} \right] \geq 1 - 2^{-\Omega(2^{n/2})}.$$

Proof. Set $U = 2^{n/2}$. The preceding counting argument bounds the number of powers with circuits of size at most $\lfloor cU/n \rfloor$ by $2^{\alpha U}$, for some fixed $\alpha < 1$ after decreasing c. On the other hand, $D_B \geq 2^{U-1}/U$. Divide the two counts to bound the exceptional fraction by $U 2^{1-(1-\alpha)U} = 2^{-\Omega(U)}$. □

The probability concerns exponent residues modulo the whole group order. It does not concern short exponent descriptions or average-case hardness over inputs to any one power. In particular, easy powers remain among the exceptional residues.

Corollary 8.6. *For all sufficiently large even n, some power of P_n has an output coordinate of circuit complexity $\Omega(2^{n/2}/n^2)$.*

Proof. For the power supplied by Theorem 8.4, join optimal circuits for all n output coordinates. Their total size is at most n times the largest coordinate complexity and is an upper bound on the complexity of the whole map. □

8.2 Why the lower bound does not apply to short input times

The lower bound selects T from $\{0, \ldots, D_B - 1\}$; it places no polynomial bound on the bit length of that exponent. Indeed, Lemma 8.2 shows that $\log D_B \geq 2^{n/2} - O(n)$. The choice of a power therefore has exponentially many available bits. The theorem does not provide a polynomial-time procedure that, given n, identifies a hard power.

For the very same family, the iterator with an m-bit time input has size $O(n(m+1))$. On an active cycle,

$$P_n^t(a, b) = (a, (b + (t \bmod a)) \bmod a). \tag{8.8}$$

An m-stage remainder computation with a k-bit running remainder costs $O(mk)$ gates, and the remaining comparison, addition, and selection cost $O(n)$ gates as in Theorem 8.1. On inactive states the answer is the input itself; use a dummy positive modulus when $a = 0$. Consequently, for $m = \mathrm{poly}(n)$ this iterator has polynomial size.

Corollary 8.7 (Bit length of hard exponents)**.** *Let T_n be exponents satisfying $\mathrm{C}(P_n^{T_n}) = \Omega(2^{n/2}/n)$ for sufficiently large even n. Their binary lengths $m_n = \max\{1, \lceil \log_2(T_n + 1) \rceil\}$ satisfy*

$$m_n = \Omega(2^{n/2}/n^2).$$

Proof. Hardwire the m_n-bit exponent into the iterator just described. This gives $\mathrm{C}(P_n^{T_n}) = O(n(m_n + 1))$. Comparing with the assumed lower bound proves the claim. □

Thus the hard powers necessarily require exponentially long binary exponents; long descriptions are not merely allowed by the counting proof. Theorem 8.4 establishes sharpness for arbitrary fixed powers. It gives no hardness result for the bounded-bit-time problem.

9 Permutation inversion and one-way functions

9.1 Inversion as a fixed power

Corollary 9.1 (Exact permutation inversion)**.** *Let $P : \{0,1\}^n \to \{0,1\}^n$ be a permutation with $\mathrm{C}(P) \le s$, and put $N = 2^n$ and $A = s + n^3$. For every $1 \le L \le N$,*

$$\mathrm{C}(P^{-1}) = O\Big(Ls + n^3\left(L + \frac{N}{L}\right)\Big).$$

In particular,

$$\mathrm{C}(P^{-1}) = O\Big(A + \sqrt{n^3 N A}\Big).$$

If $s = n^{O(1)}$, an inverter of size $2^{n/2+O(\log n)}$ is correct on every input.

Proof. Let $D = \mathrm{ord}(P)$, the least common multiple of its cycle lengths. Then P^D is the identity, so $P^{-1} = P^{D-1}$. Apply Theorem 1.1. The exponent may have a long binary expansion, but the theorem charges only the resulting tables and their access gates. □

This recovers the permutation-inversion circuit bound already discussed by Hirahara, Ilango, and Williams [4, Section 3.2]. It identifies an earlier special case of the fixed-power theorem.

9.2 Compatibility with one-wayness

A cryptographic one-way function is computable in polynomial time, while every probabilistic polynomial-time inverter succeeds with only negligible probability on the image of a uniform input. One may instead require security against every polynomial-size circuit family, a stronger nonuniform notion [6, Section 7.1.1]. For a length-preserving family $f_n : \{0,1\}^n \to \{0,1\}^n$, the latter requires that, for every such family C_n,

$$\Pr_{x \leftarrow \{0,1\}^n}\big[f_n(C_n(f_n(x))) = f_n(x)\big] \tag{9.1}$$

be negligible in n: smaller than n^{-c} eventually for every fixed $c > 0$. Any preimage counts as success. Deterministic circuits suffice in this definition, since random coins can be fixed to maximize average success. A one-way permutation is the bijective special case.

The guarantee in Corollary 9.1 is an exponential size upper bound. It therefore does not contradict either polynomial-time or polynomial-size one-wayness. Nonuniformity alone is not the reason for compatibility: Equation (9.1) already allows arbitrary nonuniform advice encoded in polynomial-size circuits. The relevant distinction is the circuit size. Moreover, no efficient procedure for constructing our inverter is asserted.

There is a quantitative consequence. For every fixed $\varepsilon > 0$, every polynomial-size permutation family on n bits has exact inverters of size at most $2^{(1/2+\varepsilon)n}$ for all sufficiently large n. Thus such a family cannot remain one-way against all circuits of that size, with any success bound strictly below one. This upper bound leaves open security against polynomial-size circuits, and does not exclude security against size 2^{cn} for a fixed $0 < c < 1/2$. It does not establish that either kind of security is achievable.

9.3 General functions and the counting example

For a noninjective map, inversion is not generally a forward power. For example, let f on $\{0,1,2,3\}$ have values $(0,0,1,0)$. The unique preimage of 1 is 2, whereas every forward iterate of 1 belongs to $\{0,1\}$. Thus Theorem 1.1 does not directly provide square-root-size inverters for arbitrary one-way functions. For comparison, Hirahara, Ilango, and Williams prove a general inversion bound of $2^{4n/5}\,\mathrm{poly}(s+n)$ gates [4, Theorem 3.5].

Encoding exhaustive preimage search as iteration does not avoid the state accounting. The straightforward transition retains an n-bit target y, an n-bit candidate x, and a flag; it tests $f(x)=y$ and either halts or increments x. This uses $2n+O(1)$ state bits. For polynomial-size f, our theorem then gives size $2^{n+O(\log n)}$, not the half exponent measured in the preimage length.

The counting construction in Section 8 also does not establish one-way functions. Its update permutation P_n has an $O(n)$-gate inverse: on an active cycle decrement b modulo a, and leave inactive states fixed. A family of the selected hard powers has exponential worst-case *forward* circuit complexity, so it cannot be a polynomial-time one-way function. The counting proof supplies no average-case inversion hardness. The results here consequently neither prove nor refute the existence of one-way functions or one-way permutations.

10 Lean formalization

The accompanying `lean/` project checks actual Boolean circuits, their evaluation semantics, and their gate bounds in Lean 4. Its coverage includes the fixed-power, invariant-set, and image-sensitive tradeoffs; real square-root optimization; permutation inversion; variable-time and batched iteration; cycle labels; transient depth; pair-input reachability and first hitting times; recognizable targets; complete-configuration simulation; eventual synchronous coalescence; recurring and permanent target predicates; and the matching-exponent lower bound. Sparse-counter centralizers and signed fixed powers are also checked, including linear Boolean gate accounting. The proofs use CSLib's circuits and simultaneous synthesis relation, Mathlib's finite dynamics and arithmetic, and the verified sorting and circuit-counting results in the author's Circuit library, distributed through `complexitylib` [10, 11, 12]. The project pins Lean and all three libraries; `lean/README.md` gives exact revisions, theorem names, and build commands.

Theorem 10.1 (Mechanized tradeoff)**.** *Let c be a circuit over the Boolean constant, NOT, AND, and OR basis, with n inputs and n outputs, and let $S \subseteq \{0,1\}^n$ be finite and invariant under its evaluation F. For all natural numbers T and $1 \le L \le 2^n$, Lean proves the existence of a circuit d agreeing with F^T on S such that*

$$|d| \le 40000\left[L|c| + (n+1)^3\left(L + \lfloor |S|/L \rfloor + 1\right)\right].$$

Here circuit size counts every gate, including constant gates.

The harmless +1s cover zero-width and empty-set cases. Define

$$\widehat{A} = s + (n+1)^3, \qquad \widehat{B} = \widehat{A} + \sqrt{(n+1)^3 2^n \widehat{A}}.$$

The checked real optimization gives the fixed-power bound $120000\widehat{B}$, and a separate theorem derives the $2^{\lceil n/2 \rceil}\,\mathrm{poly}(n)$ consequence for polynomial-size updates. The constants are deliberately loose accounting bounds. Free wiring and arbitrary fan-out permit sharing; constants are charged. Translating the paper's fixed fan-in-two basis to this basis changes size by an absolute factor.

Proof organization and shared tables. `Statement.lean` gives the foundational statement boundary using only CSLib and Mathlib. The modules in `Proofs/` build the circuits, and `Main.lean` exports the results and guards their axiom lists. The checkpoint proof packs disjoint simple paths, as in Lemma 3.1, and charges one copy of the transition circuit per retained orbit step.

For a single fixed power, one useful collision suffices. The checked implementation sorts checkpoint records, prefix records, and tagged padding. An adjacent equal-key pair with distinct nonpadding records gives either a checkpoint hit or a repeated prefix pair. Comparators share their control bit across all payload bits. Batcher networks cost $O(rR\log^2 R)$ gates for R records of r bits. The batched construction additionally checks a full shared dictionary lookup, including repeated queries and absent-key flags. It charges the checkpoint table once for the entire batch, yielding Equations (5.2) and (5.3). Conditional binary powers give both single-input and independently timed batched iterators. The long-time construction in `LongTime.lean` computes a cycle length once, uses the streaming remainder circuit to normalize the input time below $2N$, and applies binary powering to $n+3$ internal bits. Its checked optimized bound is $33000000(n+1)\widehat{B}+200m(n+1)$, proving Theorem 5.3 together with the original variable-time bound. The extra internal bits simplify overflow proofs and are fully charged.

For a repeated pair $i<j$, the fixed-power decoder starts from the hardwired residue $T \bmod (j-i)$ and adds $j-i$ until the index enters $[i,j)$. At most K corrections suffice. Width $n+2$ prevents overflow on valid witnesses, and the checked $O(Kn^2)$ cost fits the main budget. No gate processes the full binary expansion of the fixed exponent T.

Navigation implementation. The finite-dynamics layer defines transient depth, minimal cycle length, the least cycle state, and phase, and proves the reachability and first-hitting-time identities in Section 6. These semantic lemmas apply to finite types in arbitrary universes; canonical representatives additionally use a linear order.

The circuit for cycle labels uses a slight variation on the proof above. First apply $F^{N!}$. This reaches the cycle and preserves its labels, including phase, since every possible cycle length divides $N!$. On these cyclic inputs, a short prefix with no checkpoint contains the whole cycle. A scan finds its least state, its first positive return, and its first visit to that least state. Otherwise the shared dictionary supplies a checkpoint label and modular subtraction corrects its phase. This yields the checked bound $30120000\widehat{B}$.

`CheckpointDepth.lean` implements Theorem 6.2: it retains both short orbits, selects their first equality when present, and otherwise adds the prefix index to a checkpoint's stored depth. Its checked bound is $3000000\widehat{B}$. The earlier binary-search implementation is retained as a separate, weaker theorem. Two depth records and two cycle-label records feed the transient/cyclic case distinction from Theorem 6.3. The checked navigation circuit has $2n$ input bits, an existence flag, and $n+1$ time bits, with size at most $300000000(n+1)\widehat{B}$. On positive instances its decoded time is proved to be the first hit and strictly less than 2^n; its top time bit can therefore be discarded. No time-field specification is imposed on negative instances. Intermediate label and navigation words have width $n+3$. A proved shift-and-subtract division circuit supplies variable remainders in $O(n^3)$ gates; all arithmetic branches are total.

`VisitScheduleCircuits.lean` appends the target's minimal return period, using zero for a transient target. Its descriptor has one success bit and two $(n+1)$-bit fields, with checked size $360000000(n+1)\widehat{B}$. The specification describes every visit time and proves the displayed counting formula in Corollary 6.4. `VisitSchedules.lean` proves the progression identity and deadline test from a first visit on any state type, even an infinite one; `VisitCounts.lean` counts the bounded progression, including period zero. These counting identities are semantic consequences of the

record, not a gate bound for an additional deadline-processing circuit.

Applications and lower bound. The inversion proof uses $P^{N!-1} = P^{-1}$, also for encoded finite permutations. Absorbing target states give Corollary 6.5; absorption and termination hypotheses give Corollary 6.6, with the complete configuration counted in the state width. `TargetTimes.lean` proves that absorption turns the first successful visit time into transient depth. It supplies a target-existence flag and first time with bound $3120000\widehat{B}_P + a$, where $\widehat{B}_P = \mathrm{optimizedBudget}(n, s + a + 4n, 2^n)$. The checked first-halting-time circuit has bound $3000000\widehat{B}$ under semantic absorption and termination hypotheses, without requiring a recognizer circuit. Both time specifications are bounded by N; the target-time field is unspecified on negative instances.

The cycle-projection layer proves idempotence, Equation (6.2), and the finite-state equivalence used in Corollary 6.7. The checked coalescence circuit costs at most $240000\widehat{B} + 6n + 1$ gates. The recurring-target circuit composes the factorial projection with absorbing-target reachability; Boolean complements give permanent eventual acceptance. Their optimized bounds are checked with the predicate and its complement charged explicitly.

The lower-bound development constructs the modulus-counter permutation as a native Boolean circuit of size at most $50(k+1)$ on $2k$ bits. It proves injectivity of its powers below $\mathrm{lcm}(1, \ldots, 2^k - 1)$, applies Mathlib's exponential lower bound for this least common multiple, and counts circuit descriptions including output wires using the Circuit library. For every $k \geq 64$, it proves that some such power requires more than $\left\lfloor 2^k/(1024(k+1)) \right\rfloor$ gates. Thus both the small update circuit and the hard-power conclusion of Theorem 8.4 are checked. The exponent is existential; this is not a lower bound for a specified short exponent. The theorem `almost_every_counter_power` checks a stronger count: the number of easy residues, multiplied by $2^{2^{k-1}}$, is at most the period. This division-free statement proves the exceptional fraction bound in Theorem 8.5 for the native gate threshold. No probability-space infrastructure is needed for this finite uniform count.

The two native translations in `CounterRemainders.lean` have costs $s + 1$ from a power circuit to a remainder circuit, and $s + 100(k+1)$ in the opposite direction. They check the inactive branches and the spare carry bit explicitly. Hardwiring an exponent $T < 2^m$ then gives a power circuit with at most $110(m+1)(k+1)$ gates. The checked theorem `hard_power_exponent_capacity` shows that any such exponent meeting the lower-bound threshold satisfies

$$2^k < 112640(m+1)(k+1)^2.$$

This supplies a quantitative native-circuit version of Corollary 8.7 without fixing a convention for the bit length of zero.

Sparse-counter implementation. The universe-polymorphic commutation lemma proves that the defect set lies in $E \cup H^{-1}(E)$; on a finite type, injectivity of H gives the factor-two cardinality bound. For binary words, a sparse telescoping sum reconstructs displacement. Each threshold correction uses an unsigned comparison, a word selector, and a shared-carry adder. The resulting native circuit has at most $100(n+1)(r+1)$ gates. The theorem `sparse_counter_centralizer` requires injectivity of H and commutation with F, while `sparse_counter_power` specializes to a permutation and any integer exponent, including negative powers. The theorem `sparse_coordinate_power` checks the charged fixed-power transport through supplied coordinate circuits. The streaming-remainder circuit has bound $2 + m(51(k+1) + 4)$ for an m-bit numerator and a nonzero k-bit modulus, with a spare internal bit. Its semantics and cost are both checked. The arc-chart live-time theorem, sparse-transition counting application, and finite-field corollary have prose proofs; their full circuit bounds are not yet checked in Lean.

Universes and representations. The packing and reconstruction lemmas quantify over $\alpha : \mathrm{Type}\ u$ for arbitrary u; only the relevant invariant subset must be finite. The theorem `fixed_power_encoded` transports the circuit bound to a finite invariant set $S \subseteq \alpha$ with an encoding $e : \alpha \to \{0,1\}^n$ injective on S, provided $c(e(x)) = e(F(x))$ for $x \in S$. It computes $e(F^T(x))$ on encoded starting states and counts $|S|$, retaining the actual bit width n. The encoded inversion theorem assumes an injective encoding of the whole finite permutation domain. Navigation circuit bounds are stated for bit states; their underlying finite-dynamics identities are universe polymorphic. Arbitrary operations on α are not unit-cost gates, and separate initialization or decoding circuits must be charged.

Checked boundary. The principal public results have guarded axiom lists containing only `propext`, `Classical.choice`, and `Quot.sound`. There are no `sorry` placeholders, additional axioms, or native evaluation shortcuts. Noncomputable choices select checkpoints, enumerations, and circuit witnesses. This is an existence formalization, not an extracted compiler or a preprocessing-time guarantee. The sharper active-trajectory parameter, affine-controller proposal, and AKS improvement are not separately mechanized. Nor does kernel checking establish historical priority or that a formal statement captures every intended circuit convention.

11 Scope and remaining questions

The main theorem is an exact nonuniform upper bound for unrestricted bounded-fan-in circuits. It applies to noninvertible maps and is uniform in the starting state, but not in the circuit construction or its preprocessing cost. It gives no depth bound and no efficient procedure for discovering checkpoints or their payloads from a succinct representation of F. An initial-state-dependent family of update rules must be encoded as a single common transition map before the theorem applies; any additional persistent state changes n and hence the exponential scale of the bound.

The sharpness result deliberately separates the simplicity of an update rule from the description complexity of a chosen power. A stronger lower bound for a specified iteration count, or for an iterator with an n-bit time input, is not established here. The explicit permutation used for counting cannot witness such a strengthening, by Equation (8.8) and Corollary 8.7. The inversion consequence in Corollary 9.1 likewise leaves polynomial-size one-wayness open.

The sparse-counter construction gives one answer to two structural questions: it improves the dependence on transition complexity, and its chart can be compiled from edited successors without exploring the full state space. More general classes admitting these improvements remain to be identified. In particular, an additive bound $s + \widetilde{O}(2^{n/2})$ for unrestricted updates is not established. The central conclusion is the unconditional total-gate tradeoff: a short transition circuit can be combined with a sparse table, whose access cost is explicitly accounted for, to compute every fixed iterate at square-root state-space scale.

Provenance and draft status

This draft and the underlying argument were developed with assistance from ChatGPT (GPT-6 Astra Pro), including assistance with theorem formulation, proof development, exposition, and literature searches. The Lean development and the current revision also received assistance from Codex, including circuit synthesis proofs, axiom auditing, and review. The author field is provisional. Historical priority and final attribution require author review; the draft does not represent independent peer verification.

References

[1] Kenneth E. Batcher. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, AFIPS Conference Proceedings, volume 32, pages 307–314, 1968. doi:10.1145/1468075.1468121.

[2] Martin E. Hellman. A cryptanalytic time–memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980. Author-hosted full text.

[3] J. Ian Munro, Rajeev Raman, Venkatesh Raman, and S. Srinivasa Rao. Succinct representations of permutations and functions. *Theoretical Computer Science*, 438:74–88, 2012. Preprint: arXiv:1108.1983, 2011.

[4] Shuichi Hirahara, Rahul Ilango, and R. Ryan Williams. Beating brute force for compression problems. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, pages 659–670, 2024. doi:10.1145/3618260.3649778. Full version: ECCC TR23-171; section and theorem numbers refer to that version.

[5] David Eppstein. The complexity of iterated reversible computation. *TheoretiCS*, 2, Article 10, 1–41, 2023. doi:10.46298/theoretics.23.10.

[6] Oded Goldreich. *Computational Complexity: A Conceptual Perspective*. Cambridge University Press, 2008. Author's draft, Chapter 7: one-way function definitions.

[7] Travis Gagie, Meng He, and Gonzalo Navarro. Path queries on functions. *Theoretical Computer Science*, 770:34–50, 2019. doi:10.1016/j.tcs.2018.10.021. Author-hosted full text.

[8] Rutilo Moreno and Luis Manuel Rivera. Blocks in cycles and k-commuting permutations. *SpringerPlus*, 5, article 1949, 2016. Theorem numbers refer to arXiv:1306.5708v3 (2014), `https://arxiv.org/abs/1306.5708v3`.

[9] Thorsten Kleinjung and Benjamin Wesolowski. Discrete logarithms in quasi-polynomial time in finite fields of fixed characteristic. *Journal of the American Mathematical Society*, 35(2):581–624, 2022. Full text: Cryptology ePrint Archive, Paper 2019/751, `https://eprint.iacr.org/2019/751`.

[10] The CSLib contributors. CSLib: a Lean library for computer science. `https://github.com/leanprover/cslib`. Pinned revisions are in the accompanying Lean dependency manifest.

[11] The mathlib community. The Lean mathematical library. In *Proceedings of CPP*, pages 367–381, 2020. doi:10.1145/3372885.3373824. Lean 4 implementation: `https://github.com/leanprover-community/mathlib4`.

[12] Samuel Schlesinger and contributors. Complexitylib, including the Circuit library and verified sorting networks. `https://github.com/SamuelSchlesinger/complexitylib`.